



CHALMERS
UNIVERSITY OF TECHNOLOGY

CA2: Kirchhoff Plate Analysis

TME245 - Finite Element Method: Structures

Academic year 2024/2025, SP3

Authors: Group 27

Paulo Liborio, librio@chalmers.se

Sankalp Manojkumar Prabhu, sankalpm@chalmers.se

March 6, 2025

Contents

1	Task 1: Linear Elastic Plate Analysis with Matlab	1
1.1	Task 1A - Problem Description	1
1.2	Task 1B - Kirchhoff Plate Element	3
1.2.1	FE Formulation:	3
1.2.2	Element Routine	5
1.2.3	External Force Contribution	6
1.3	Task 1C - FEA of Plate Structure	7
1.4	Task 1D - Stress Computation	8
1.5	Results and Discussion	10
2	Linearized Buckling using Matlab	11
2.1	Task 2A - Element Routine	11
2.2	Task 2B - Pre-Buckling Analysis	13
2.3	Results and Discussion	14
	References	14
A	Appendix: Code listing	I
A.1	Task 1B, Kirchhoff Element Routine	I
A.2	Task 1C, Solve for Displacements	III
A.3	Task 1D, Stress Function	VIII
A.4	Task 2A, Geometric Stiffness Matrix Function	IX
A.5	Task 2B, Buckling Problem	X
B	Appendix: Support Code Listing	XV
B.1	Kirchhoff Bending B-Matrix	XV
B.2	Quad Element B-Matrix	XVII

1

Task 1: Linear Elastic Plate Analysis with Matlab

1.1 Task 1A - Problem Description

A schematic representation of the pool wall section is provided in Figure 1.1, defining the coordinate system and boundary conditions. All edges are clamped, with the exception of the top one, which is free. The pool wall is subjected to a hydrostatic pressure $q(y) = \rho g \bar{y}$. An in-plane traction load is applied to a section of the top edge, as a representation of a person with mass m standing on a platform. Relevant variables are listed:

$t = 10mm$	(Wall Thickness),	$\rho = 1000kg/m^3$	(Water Density).
$E = 210GPa$	(Young's Modulus),	$g = 9.81m/s^2$	(Gravity).
$\sigma_y = 450MPa$	(Yield Strength),	$m = 150kg$	(Person's mass).
$\nu = 0.3$	(Poisson's Ratio),	$FS \geq 2$	(Factor of Safety).

The problem is analyzed through the Finite Element approximation of Kirchhoff plates. For that, the first step is to define the problem in the Weak Form, by way of the *Principle of Virtual Work*. The weak form of Kirchhoff love plate theory is divided in two parts, one for the in-plane (membrane) problem (Eq. 1.1) and another for the out-of plane (bending) problem (Eq. ??):

$$\int_A \delta \underline{\epsilon}^{0T} \underline{N}^{sec} dA = \int_A \delta \underline{u}^{0T} \underline{f}_u dA + \int_{\mathcal{L}_{g,0}} \delta \underline{u}^{0T} \underline{P} d\mathcal{L} + \int_{\mathcal{L}_{h,0}} \delta \underline{u}^{0T} \underline{P} d\mathcal{L} \quad (1.1)$$

$$\begin{aligned} - \int_A \delta \kappa_{\alpha\beta} M_{\alpha\beta} dA &= \int_A \delta w q dA - \int_{\mathcal{L}_{g,II}} \frac{\partial(\delta w)}{\partial n} M_{nn} d\mathcal{L} - \int_{\mathcal{L}_{h,II}} \frac{\partial(\delta w)}{\partial n} M_{nn} d\mathcal{L} \\ &+ \int_{\mathcal{L}_{g,I}} \delta w \left(V_n + \frac{\partial M_{nm}}{\partial m} \right) d\mathcal{L} + \int_{\mathcal{L}_{h,I}} \delta w \left(V_n + \frac{\partial M_{nm}}{\partial m} \right) d\mathcal{L} \end{aligned} \quad (1.2)$$

Boundary conditions for the problem can be expressed as:

- $u_x = u_y = 0$ on $\mathcal{L}_{g,0}$,
- $w = 0$ on $\mathcal{L}_{g,I}$,
- $\frac{\partial w}{\partial n} = 0$ on $\mathcal{L}_{g,II}$,
- $P = \bar{P}$ on $\mathcal{L}_{h,0}$.
- $V_n + \frac{\partial M_{nn}}{\partial m} = V^k = 0$ on $\mathcal{L}_{h,I}$.
- $M_{nn} = 0$ on $\mathcal{L}_{h,II}$.

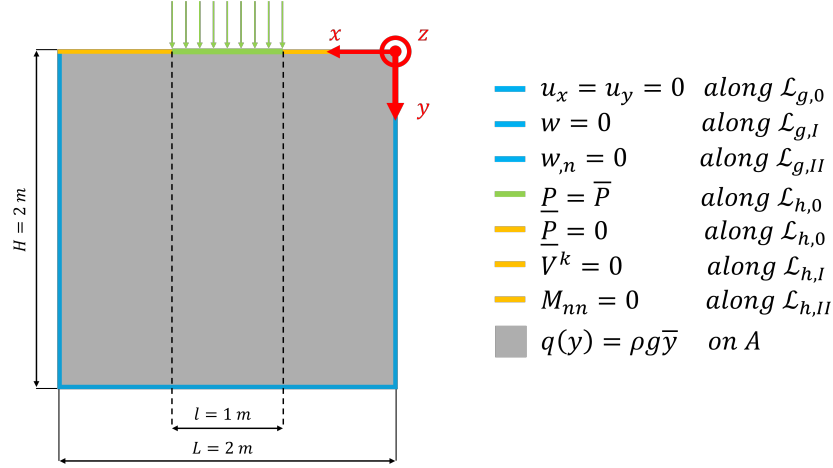


Figure 1.1: Boundary Conditions Visualized

The Dirichlet Boundary conditions are enforced by prescribing the displacement or the rotation value to the boundary degrees of freedom which means removal of those DoFs from the stiffness matrix and force vector for FE formulation.

The Neumann Boundary condition is enforced by incorporating the described values in the force vector for FE formulation.

After applying boundary conditions to the weak form, Equations 1.1 & ?? can be re-written as:

$$\int_A \delta \underline{\epsilon}^{0T} \underline{N}^{sec} dA = \underbrace{\int_A \delta \underline{u}^{0T} \underline{f}_u dA}_{\text{I}} + \underbrace{\int_{\mathcal{L}_{g,0}} \delta \underline{u}^{0T} \underline{P} d\mathcal{L}}_{\text{II}} + \int_{\mathcal{L}_{h,0}} \delta \underline{u}^{0T} \underline{P} d\mathcal{L} \quad (1.3)$$

In Equation 1.3, term **I** is eliminated as body load is not considered for this assignment. Term **II** is also eliminated as the edge $\mathcal{L}_{g,0}$ is clamped, i.e., in plane displacement is set to zero.

$$\begin{aligned} - \int_A \delta \kappa_{\alpha\beta} M_{\alpha\beta} dA &= \int_A \delta w q dA - \underbrace{\int_{\mathcal{L}_{g,II}} \frac{\partial(\delta w)}{\partial n} M_{nn} d\mathcal{L}}_{\text{I}} - \underbrace{\int_{\mathcal{L}_{h,II}} \frac{\partial(\delta w)}{\partial n} M_{nn} d\mathcal{L}}_{\text{II}} \\ &\quad + \underbrace{\int_{\mathcal{L}_{g,I}} \delta w \left(V_n + \frac{\partial M_{nn}}{\partial m} \right) d\mathcal{L}}_{\text{III}} + \underbrace{\int_{\mathcal{L}_{h,I}} \delta w \left(V_n + \frac{\partial M_{nn}}{\partial m} \right) d\mathcal{L}}_{\text{IV}} \end{aligned} \quad (1.4)$$

In Equation 1.4, terms **I** and **III** are eliminated as edges $\mathcal{L}_{g,I}$ and $\mathcal{L}_{g,II}$ are clamped, i.e., out of plane displacements and rotations are set to zero. Terms **II** and **IV** are also eliminated as $\mathcal{L}_{h,I}$ and $\mathcal{L}_{h,II}$ are the free edges, which have no prescribed moments (M_{nn}) and Kirchhoff shear forces (V^K).

1.2 Task 1B - Kirchhoff Plate Element

1.2.1 FE Formulation:

The Kirchhoff plate element formulation considers both in-plane and out of plane deformation, analyzing bending and membrane effects independently, that is, the problems are decoupled. The total stiffness matrix and external force contributions are derived from weak form expressed as:

$$\begin{bmatrix} \underline{K}_{uu} & 0 \\ 0 & \underline{K}_{ww}^K \end{bmatrix} \begin{bmatrix} \underline{a}_u \\ \underline{a}_w^K \end{bmatrix} = \begin{bmatrix} \underline{f}_u^{ext} \\ \underline{f}_w^K \end{bmatrix} \quad (1.5)$$

Where:

1. $\underline{K}_{uu}$ is the membrane stiffness matrix which can be expressed as

$$\underline{K}_{uu} = \int_A \underline{B}_u^T \underline{D}^0 \underline{B}_u dA \quad (1.6)$$

2. $\underline{a}_u$ and $\underline{a}_w^K$ are the in-plane and out-of-plane displacement vectors, respectively:

$$\underline{a}_u = \begin{bmatrix} a_x^1 \\ a_y^1 \\ a_x^2 \\ a_y^2 \\ \vdots \\ a_y^n \end{bmatrix}, \quad \underline{a}_w^K = \begin{bmatrix} w^1 \\ \theta_x^1 \\ \theta_y^1 \\ w^2 \\ \vdots \\ \theta_y^n \end{bmatrix} \quad (1.7)$$

3. $\underline{K}_{ww}^K$ is the bending stiffness matrix which is expressed as

$$\underline{K}_{ww}^K = \int_A \underline{\tilde{B}}^* \underline{\tilde{D}}^* \underline{\tilde{B}} dA \quad (1.8)$$

4. $\underline{f}_u^{ext}$ is the in-plane traction which is expressed by modifying the weak formulation from equation ?? as

$$\underline{f}_u^{ext} = \int_{\mathcal{L}_{h,0}} \underline{N}_u^T \underline{P} d\mathcal{L} \quad (1.9)$$

5. $\underline{f}_w^{K,ext}$ is the out-of plane load contribution on the plate which can be expressed by modifying the weak formulation from equation ?? as

$$\underline{f}_w^K = \int_A \underline{N}^T q dA \quad (1.10)$$

To transition from the weak form to FE-approximation, a few key terms were introduced: membrane strain-displacement matrix ($\underline{B}_u$), bending strain-displacement matrix ($\underline{\tilde{B}}^*$), Shape Functions ($\underline{N}_u$) and Kirchhoff Shape Function ($\underline{N}$) which can be expressed as:

- Shape Function

$$\underline{N}_u = \begin{bmatrix} N^1 & 0 & N^2 & 0 & \cdot & \cdot & N^{n_{node}} & 0 \\ 0 & N^1 & 0 & N^2 & \cdot & \cdot & 0 & N^{n_{node}} \end{bmatrix} \quad (1.11)$$

- Membrane strain-displacement matrix:

$$\underline{B}_u = \underbrace{\begin{bmatrix} \partial/\partial x & 0 \\ 0 & \partial/\partial y \\ \partial/\partial y & \partial/\partial x \end{bmatrix}}_{\underline{\tilde{V}}} \underline{N}_u$$

$$\underline{B}_u = \begin{bmatrix} \partial N^1/\partial x & 0 & . & . & \partial N^{n_{node}}/\partial x & 0 \\ 0 & \partial N^1/\partial y & . & . & 0 & \partial N^{n_{node}}/\partial y \\ \partial N^1/\partial y & \partial N^1/\partial x & . & . & \partial N^{n_{node}}/\partial y & \partial N^{n_{node}}/\partial x \end{bmatrix} \quad (1.12)$$

- Kirchhoff Shape Functions:

$$\underline{N} = \begin{bmatrix} N^1 & N^2 & N^3 & . & . & N^{ndof} \end{bmatrix} \quad (1.13)$$

This shape function has a particular formulation:

$$w(\xi, \eta) = \alpha_1 + \alpha_2 \xi + \alpha_3 \eta + \alpha_4 \xi^2 + \alpha_5 \xi \eta + \alpha_6 \eta^2 + \alpha_7 \xi^3 + \alpha_8 \xi^2 \eta + \alpha_9 \xi \eta^2 + \alpha_{10} \eta^3 + \alpha_{11} \xi^3 \eta + \alpha_{12} \xi \eta^3$$

which in matrix form can be written as:

$$w = \underline{P}(\xi, \eta) \underline{\alpha}$$

The expression of shape function is derived from the imposed constraint:

$$w^1 = [\underline{P}]_{(-1,-1)} \underline{\alpha}$$

$$\theta_x^1 = \left[\frac{\partial \underline{P}}{\partial \xi} \frac{\partial \xi}{\partial x} \right]_{(-1,-1)} \underline{\alpha}, \quad \theta_y^1 = \left[\frac{\partial \underline{P}}{\partial \eta} \frac{\partial \eta}{\partial x} \right]_{(-1,-1)} \underline{\alpha}, \quad \dots \quad \theta_y^4 = \left[\frac{\partial \underline{P}}{\partial \eta} \frac{\partial \eta}{\partial x} \right]_{(-1,1)} \underline{\alpha}$$

or $\underline{a}_w^{K,e} = \underline{A} \underline{\alpha}$ whereby $\underline{\alpha} = \underline{A}^{-1} \underline{a}_w^{K,e}$.

w can be rewritten as:

$$w = \underline{P} \underline{A}^{-1} \underline{a}_w^{K,e}$$

Therefore,

$$\underline{N} = \underline{P} \underline{A}^{-1}$$

- Bending-strain displacement matrix:

$$\underline{\overset{*}{B}} = \underbrace{\begin{bmatrix} \frac{\partial^2}{\partial x^2} \\ \frac{\partial^2}{\partial y^2} \\ 2 \frac{\partial^2}{\partial x \partial y} \end{bmatrix}}_{\underline{\overset{*}{V}}} \underline{N} \rightarrow \underline{\overset{*}{B}} = \begin{bmatrix} \frac{\partial^2 N^1}{\partial x^2} & \frac{\partial^2 N^2}{\partial x^2} & . & . & \frac{\partial^2 N^{ndof}}{\partial x^2} \\ \frac{\partial^2 N^1}{\partial y^2} & \frac{\partial^2 N^2}{\partial y^2} & . & . & \frac{\partial^2 N^{ndof}}{\partial y^2} \\ 2 \frac{\partial^2 N^1}{\partial x \partial y} & 2 \frac{\partial^2 N^2}{\partial x \partial y} & . & . & 2 \frac{\partial^2 N^{ndof}}{\partial x \partial y} \end{bmatrix} \quad (1.14)$$

1.2.2 Element Routine

To ensure that the computation of the Kirchhoff plate element is correct, the element routine was verified using the test case. The verification process involved the computing element level membrane and bending stiffness matrix as well as applied external loads. The element outputs are shown below:

External applied force:

$$f_w^{e,ext} = \begin{bmatrix} 0.0196 \\ 0 \\ 0 \\ 0.0186 \\ 0 \\ 0 \\ 0.0187 \\ 0 \\ 0 \\ 0.0197 \\ 0 \\ 0 \end{bmatrix}$$

Membrane Stiffness matrix:

$$K_{uu}^e = 10^9 \times \begin{bmatrix} 2.7986 & 0.5347 & -2.4879 & -0.1154 & -1.4127 & -0.5787 & 1.1021 & 0.1595 \\ 0.5347 & 1.626 & 0.3012 & -0.6957 & -0.5787 & 0.7750 & -0.2572 & -0.1553 \\ -2.4879 & 0.3012 & 3.0698 & -0.7208 & 0.9301 & -0.2517 & -1.5119 & 0.6713 \\ -0.1154 & -0.6957 & -0.7208 & 1.9774 & 0.1649 & -0.2610 & 0.6713 & -1.0208 \\ -1.4127 & -0.5787 & 0.9301 & 0.1649 & 2.9515 & 0.5298 & -2.4688 & -0.1160 \\ -0.5787 & -0.7750 & -0.2517 & -0.2610 & 0.5298 & 1.7199 & 0.3006 & -0.6839 \\ 1.1021 & -0.2572 & -1.5119 & 0.6713 & -2.4688 & 0.3006 & 2.8786 & -0.7147 \\ 0.1595 & -0.1553 & 0.6713 & -1.0208 & -0.1160 & -0.6839 & -0.7147 & 1.8601 \end{bmatrix}$$

For ease of visualization the Bending stiffness matrix K_{ww}^e is divided into 4 smaller matrices with following arrangement:

$$K_{ww}^e = 10^{11} \times \begin{bmatrix} [K_{11}^e] & [K_{12}^e] \\ [K_{21}^e] & [K_{22}^e] \end{bmatrix}$$

$$[K_{11}^e] = \begin{bmatrix} 0.9973 & 0.0039 & 0.0012 & -0.9057 & 0.0036 & 0.0006 \\ 0.0039 & 0.0000 & 0.0000 & -0.0038 & 0.0000 & 0.0000 \\ 0.0012 & 0.0000 & 0.0000 & -0.0005 & 0.0000 & 0.0000 \\ -0.9057 & -0.0038 & -0.0005 & 1.0059 & -0.0037 & 0.0003 \\ 0.0036 & 0.0000 & 0.0000 & -0.0037 & 0.0000 & 0.0000 \\ 0.0006 & 0.0000 & 0.0000 & 0.0003 & 0.0000 & 0.0000 \end{bmatrix}$$

$$[K_{12}^e] = \begin{bmatrix} -0.3247 & 0.0017 & 0.0001 & 0.2330 & 0.0019 & 0.0004 \\ -0.0018 & 0.0000 & -0.0000 & 0.0016 & 0.0000 & -0.0000 \\ -0.0004 & 0.0000 & 0.0000 & -0.0004 & 0.0000 & 0.0000 \\ 0.2245 & -0.0015 & 0.0007 & -0.3247 & -0.0020 & 0.0005 \\ -0.0018 & 0.0000 & -0.0000 & 0.0019 & 0.0000 & -0.0000 \\ -0.0007 & 0.0000 & 0.0000 & -0.0002 & 0.0000 & 0.0000 \end{bmatrix}$$

$$[K_{21}^e] = \begin{bmatrix} -0.3247 & -0.0018 & -0.0004 & 0.2245 & -0.0018 & -0.0007 \\ 0.0017 & 0.0000 & 0.0000 & -0.0015 & 0.0000 & 0.0000 \\ 0.0001 & -0.0000 & 0.0000 & 0.0007 & -0.0000 & 0.0000 \\ 0.2330 & 0.0016 & -0.0004 & -0.3247 & 0.0019 & -0.0002 \\ 0.0019 & 0.0000 & 0.0000 & -0.0020 & 0.0000 & 0.0000 \\ 0.0004 & -0.0000 & 0.0000 & 0.0005 & -0.0000 & 0.0000 \end{bmatrix}$$

$$[K_{22}^e] = \begin{bmatrix} 0.9913 & -0.0037 & -0.0003 & -0.8911 & -0.0037 & 0.0003 \\ -0.0037 & 0.0000 & -0.0000 & 0.0036 & 0.0000 & -0.0000 \\ -0.0003 & -0.0000 & 0.0000 & -0.0004 & -0.0000 & 0.0000 \\ -0.8911 & 0.0036 & -0.0004 & 0.9827 & 0.0038 & -0.0012 \\ -0.0037 & 0.0000 & -0.0000 & 0.0038 & 0.0000 & -0.0000 \\ 0.0003 & -0.0000 & 0.0000 & -0.0012 & -0.0000 & 0.0000 \end{bmatrix}$$

1.2.3 External Force Contribution

The Kirchhoff plate element used in this analysis is subjected to two external loads as described in section 1.1: a hydrostatic pressure distribution due to water inside the pool and a in-plane traction force due to person standing on a platform. Both loads are incorporated into FE formulation by computing the respective external force at element level. The formulas for the same can be expressed by rewriting equations 1.9 and 1.10 as:

$$\underline{f}_u^{ext} = \int_{\mathcal{L}} \underline{N}_u^T \underline{t} d\mathcal{L} \quad \text{with} \quad \underline{t} = \begin{bmatrix} t_x \\ t_y \end{bmatrix}, \quad \underline{f}_w^K = \int_A \underline{N}^T q dA \quad (1.15)$$

$\underline{t}$ is the traction along the edge $\mathcal{L}_{h,0}$ and $q = \rho g y$ is the hydrostatic pressure, linearly varying with depth. To simplify the implementation, the pressure is assumed to be constant over each element and evaluated at its midpoint, such that q can be re-written as $q = \rho g \bar{y}_e$, where $\bar{y}_e$ is the vertical position of the element's centroid.

To verify the accuracy of the forces computed using FE-formulation, the total nodal forces are summed and compared with the expected total forces as follows:

- In-Plane Traction total force:

$$f_u^{total} = m \cdot g = 150 \cdot 9.806 = 1470.90 N$$

- Hydrostatic total force:

$$f_w^{total} = \rho g L \frac{h_{bottom}^2 - h_{top}^2}{2} = 1000 \cdot 9.806 \cdot 2 \cdot \frac{2^2}{2} = 39224.00 N$$

Where h_{top} represent the depth coordinate where the free fluid surface is and h_{bottom} represents the bottom.

These are computed in Matlab as the sum of the respective force vectors $\underline{f}_u$ and $\underline{f}_w$, denoted as f_{aa} and f_{ww} , respectively.

```

1 % Total In-Plane and Out-of-Plane Forces
2 fa_tot = sum(faa);
3 fw_tot = sum(fww);
4
5 fprintf('Total in-plane forces = %.2f [N]\n', fa_tot);
6 fprintf('Total out-of-plane forces = %.2f [N]\n', fw_tot);
7
8 % Output
9 Total in-plane forces = 1470.90 [N]
10 Total out-of-plane forces = 39224.00 [N]
```


1.3 Task 1C - FEA of Plate Structure

In this task, the pool wall section is analyzed with the Kirchhoff plate element routine implemented in Task 1B. The objective is to solve for both in-plane and out-of-plane displacements, under the combined loading condition. Since the solution for the problems is decoupled and output displacements which are up to 10^5 different in order of magnitude, the displacements are plotted separately. The complete code for this task is found at Appendix A.5.

Component	Value [mm]	Position (x, y) [m]
u_x	1.63×10^{-4}	(2.5, 0.0)
u_y	7.34×10^{-4}	(3.0, 0.0)
w	13.12	(3.0, 0.9)

Table 1.1: Summary of Von-Mises Stress and Factor of Safety

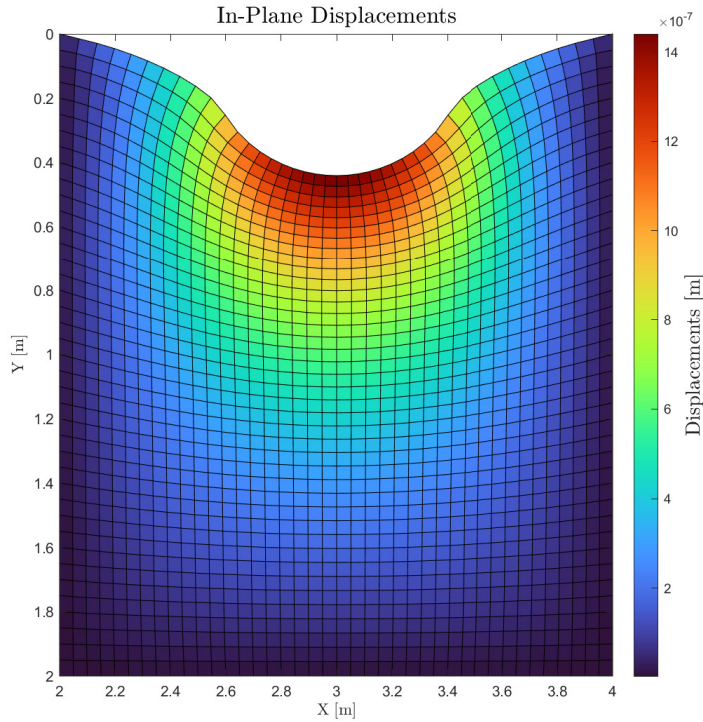


Figure 1.2 illustrates the in-plane displacement field of the pool wall section. The displacement is highest near the top edge, where the traction load is applied. A scale factor of 600000 times has been applied to improve the visualization of the deformed structure.

Figure 1.2: In-Plane Displacements

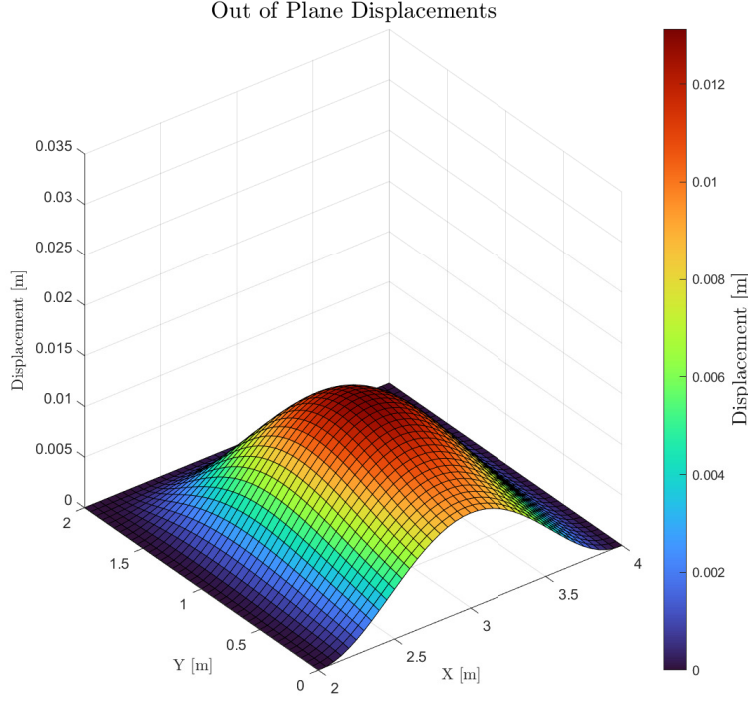


Figure 1.3 shows the out of plane deformation of the structure, as a result of the applied hydrostatic pressure. The maximum displacement happens at lower middle section, since the hydrostatic pressure increase linearly with the y -coordinate.

Figure 1.3: Out-of-Plane Displacements

1.4 Task 1D - Stress Computation

The objective of this task is to compute and analyze the in-plane stress distribution of the structure subjected to the given loads. The in-plane stress is computed at the Gauss Points of the elements with the expression as follows:

$$\underline{\sigma} = \underline{D} (\underline{B}_u^e \underline{a}_u^e - z^* \underline{B}_w^* \underline{a}_w^{K,e}) \quad (1.16)$$

To obtain a straightforward representation of the stress field, values for the elements' Gauss Points are averaged, such that each element contains only one stress tensor. From there, the Von Mises equivalent stress computed. In Voigt form, it is calculated as:

$$\underline{s} = \begin{bmatrix} \sigma_{11} \\ \sigma_{22} \\ \sigma_{33} \\ \sigma_{12} \\ \sigma_{13} \\ \sigma_{23} \end{bmatrix} - \frac{(\sigma_{11} + \sigma_{22} + \sigma_{33})}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \sigma_{vM} = \sqrt{\frac{3}{2}(\underline{s}^T \underline{s} + \sigma_{12}^2 + \sigma_{13}^2 + \sigma_{23}^2)} \quad (1.17)$$

Using the equivalent stress, the structure is assessed by a Factor of Safety, defined as the ratio of the allowable load to the acting stresses. The allowable load is defined as the materials yield strength $\sigma_y = 450\text{MPa}$, such that a factor of safety above one means the component passes with the specified design factor.

$$FS = \frac{\sigma_{yield}}{\sigma_{acting}} \quad (1.18)$$

The Von-Mises equivalent stress and safety factor are computed for the whole structure at three different positions through the thickness, at the bottom, middle and top surfaces, or at $z = [-h/2, 0, h/2]$.

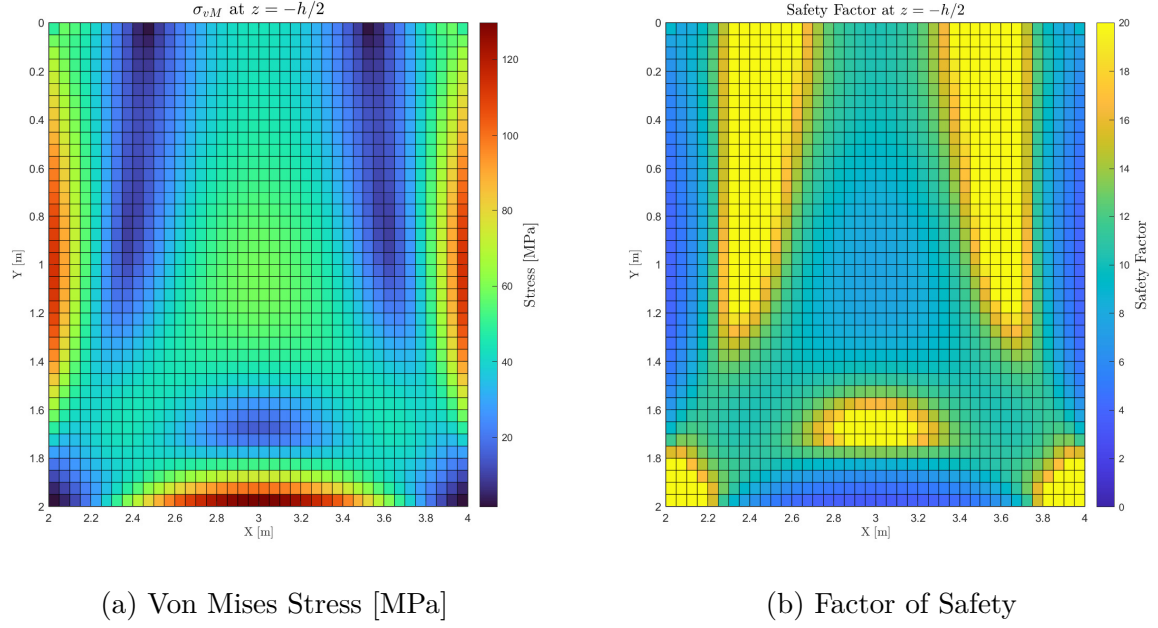


Figure 1.4: Bottom surface $z = -h/2$

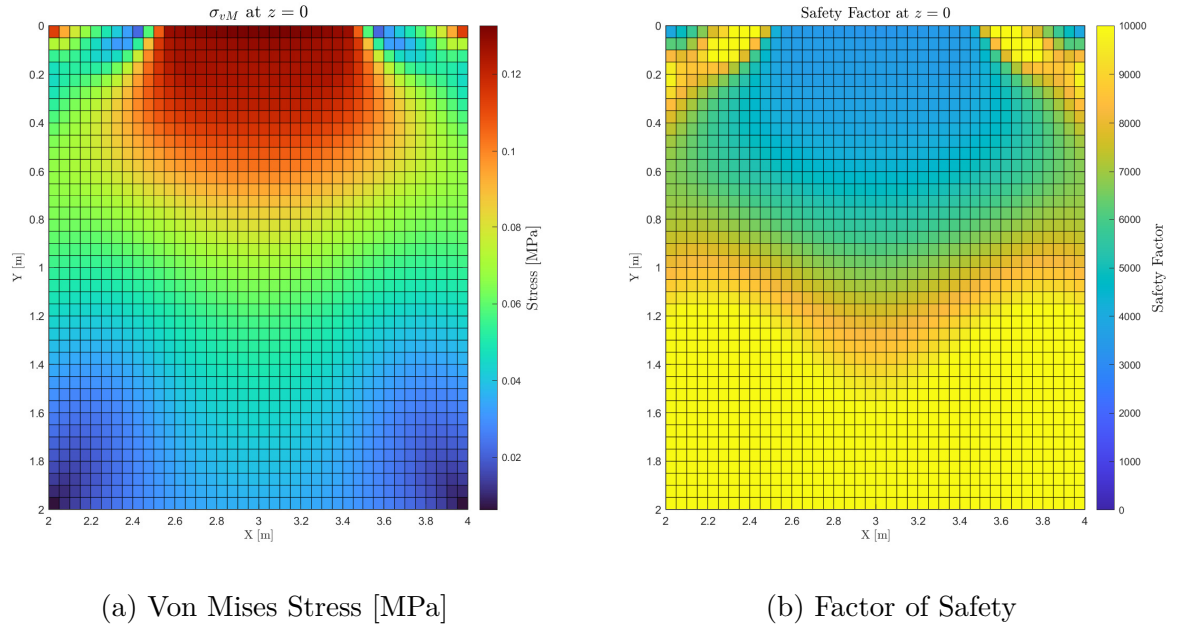
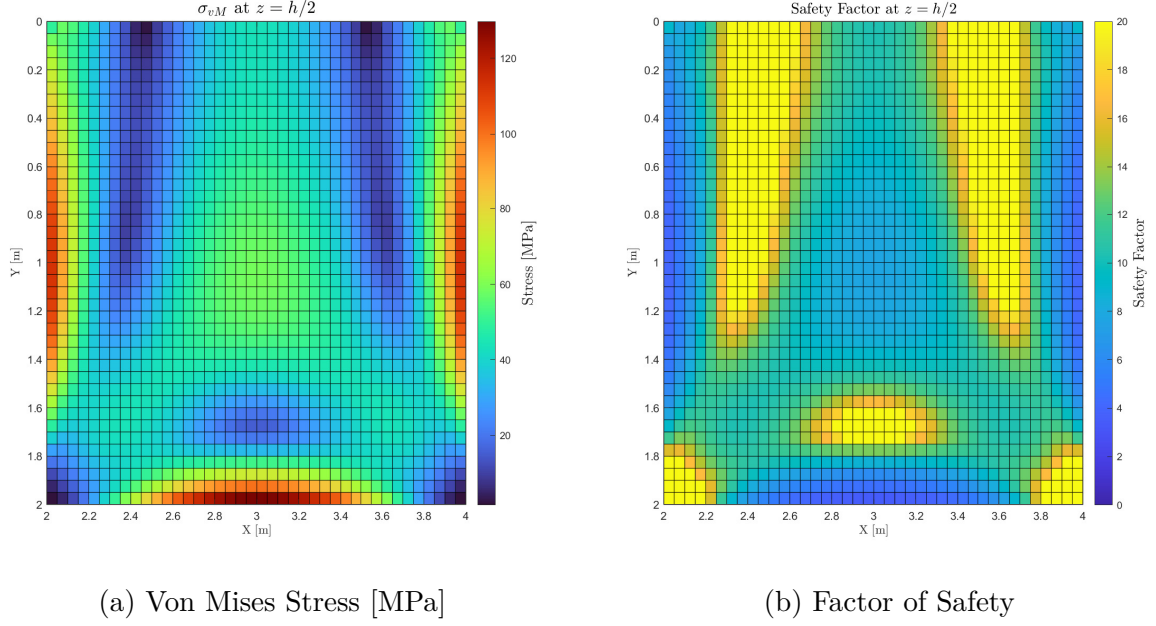


Figure 1.5: Middle surface $z = 0$


Figure 1.6: Top surface $z = h/2$

z position	Max vMises [MPa]	Min FoS
$-h/2$	129.67	3.47
0	0.13	3396
$h/2$	129.74	3.47

Table 1.2: Summary of Von-Mises Stress and Factor of Safety

1.5 Results and Discussion

From the displacement and stress field contour plots, it is evident that the structural behavior of a plate is highly dominated by the out-of-plane forces, that causes bending. This is expected, since thin-walled structures are built mainly to carry membrane loads, although they are able to sustain out-of-plane loads. To make a fair comparison between load carrying capacities - membrane vs bending - we can leverage the superposition property of a linear analysis. For this problem, $f_w^{ext} \approx 27 \times f_u^{ext}$. Applying this ratio as a load multiplier to the in-plane traction would scale the Von Mises stress by the same amount. As a result, the maximum in-plane Von Mises stress at the middle surface would be $= 3.5\text{MPa}$, a value ≈ 38 times smaller than those found at the top and bottom surfaces.

It can also be observed that the top and bottom surfaces exhibit a similar stress distribution, which is a typical behavior for plate structures under bending, such that normal stresses reach their extrema at the outermost surfaces and approach zero at the neutral axis. Meanwhile, the middle surface has its highest stresses near the point of load application. Additionally, stress concentrations occur on the edges where the boundary conditions have been defined, a characteristic behavior of FE models.

Since the Factor of Safety requirements are met for the whole structure, it is concluded that maintains structural integrity for the yield strength criterion. The next step is to assess its safety against instability, or buckling.

2

Linearized Buckling using Matlab

Plate buckling is an instability phenomenon that might occur due to compressive in-plane, or membrane, forces in a plate. The membrane forces are obtained according to 1st order theory and are used in the buckling analysis, which is way such it is known as linearized pre-buckling problem (LPB). The assumptions are linear elasticity and small strains kinematics, while the membrane forces are considered large enough to have a significant effect on the moment equilibrium - 2nd order theory.

A scalar loading parameter λ is introduced, such that the loads are comprised of the load factor λ and a reference load. The external force vector reads:

$$\underline{f}^{\text{ext}} = \lambda \left[\int_A \underline{N}^T q^{(R)} dA + \int_{\mathcal{L}} \underline{N}^T V_K^{(R)} d\mathcal{L} - \int_{\mathcal{L}} \underline{B}^T \underline{n} M_{nn}^{(R)} d\mathcal{L} \right] \quad (2.1)$$

To consider the boundary conditions in the FE-formulation, a static condensation form is assumed, splitting the solution into *Free* and *Constrained* nodal components, resulting in expression 2.2:

$$\left[\underline{K}_{wwFF}^K + \lambda \underline{G}_{FF}^{(R)} \right] \underline{a}_{w,F}^K = \lambda \underline{f}_F^{\text{ext}(R)} - \left[\underline{K}_{wwFC}^K + \lambda \underline{G}_{FC}^{(R)} \right] \lambda \underline{a}_{w,C}^{K,(R)} \quad (2.2)$$

If appropriate boundary conditions are established, ensuring that rigid body motions are prevented, $\underline{K}_{wwFF}^K$ is always positive definite. Consequently, a unique solution is assured if

$$\det \left(\underline{K}_{wwFF}^K + \lambda \underline{G}_{FF}^{(R)} \right) > 0,$$

and a critical (buckling) situation arises when

$$\det \left(\underline{K}_{wwFF}^K + \lambda \underline{G}_{FF}^{(R)} \right) = 0. \quad (2.3)$$

Therefore, the critical values of λ can be found by solving the generalized Eigenvalue Problem:

$$\underline{K}_{wwFF}^K \underline{z}_i = -\lambda_i \underline{G}_{FF}^{(R)} \underline{z}_i \quad i = 1, 2, \dots, N_F \quad (2.4)$$

2.1 Task 2A - Element Routine

The first step of a LPB is to solve the in-plane problem to obtain the reference membrane forces $\underline{\tilde{N}}^{\text{sec}}$. A quadrilateral linear element is not a constant strain element, therefore the stresses are not constant as well, which ultimately means that $\underline{\tilde{N}}^{\text{sec}}$ also varies throughout the element. Although, for simplification, the stress has been averaged between the integration points, so that the membrane forces are constant in a single element.

$$\underline{\tilde{N}}^{e,\text{sec}(R)} \approx h^e \cdot \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}^e \quad (2.5)$$

The membrane forces are used to calculate the geometric stiffness matrix $G^{e,(R)}$ for all elements, assembled into the global matrix $G^{(R)}$:

$$\underline{G}^{e,(R)} = \int_A \underline{B}^{eT} \tilde{\underline{N}}^{\text{sec}(R)} \underline{B}^e dA, \quad (2.6)$$

$$(2.7)$$

$\underline{B}^e$ is the element gradient matrix, while N^i are the *Kirchhoff plate shape functions*:

$$\underline{B}^e(x, y) = \underline{\nabla} N(x, y) = \begin{bmatrix} \frac{\partial N_1}{\partial x}(x, y) & \frac{\partial N_2}{\partial x}(x, y) & \cdots & \frac{\partial N_{12}}{\partial x}(x, y) \\ \frac{\partial N_1}{\partial y}(x, y) & \frac{\partial N_2}{\partial y}(x, y) & \cdots & \frac{\partial N_{12}}{\partial y}(x, y) \end{bmatrix} \quad (2.8)$$

It is important to note that since the Kirchhoff approximation for the out-of-plane deflection is a third-order polynomial, the Kirchhoff $\underline{B}^e$ matrix is inherently of second order. As a result, using a 2×2 Gauss integration scheme is insufficient for integrating $G_{(R)}^e$. Instead, a 3×3 Gauss integration scheme must be employed, with the following coordinates and weights in the parent domain:

ξ_1	ξ_2	H
$-\sqrt{3/5}$	$-\sqrt{3/5}$	25/81
$\sqrt{3/5}$	$-\sqrt{3/5}$	25/81
$-\sqrt{3/5}$	$\sqrt{3/5}$	25/81
$\sqrt{3/5}$	$\sqrt{3/5}$	25/81
0	$-\sqrt{3/5}$	40/81
0	$\sqrt{3/5}$	40/81
$-\sqrt{3/5}$	0	40/81
$\sqrt{3/5}$	0	40/81
0	0	64/81

Table 2.1: Gaussian Integration Scheme for a Kirchhoff Buckling Element

To assess the accuracy of the computation of the $\underline{G}^R$ matrix, a test case has been provided as reference. The output of the implemented element routine is split into four smaller matrices and printed:

$$\underline{G}_{int}^{e,(R)} = \begin{bmatrix} [G_{11}^e] & [G_{12}^e] \\ [G_{21}^e] & [G_{22}^e] \end{bmatrix}$$

$$\begin{aligned}
G_{11}^e &= 10^5 \cdot \begin{bmatrix} 1.1085 & 0.0006 & 0.0013 & -0.1916 & -0.0001 & -0.0018 \\ 0.0006 & 0.0000 & 0.0000 & 0.0001 & -0.0000 & -0.0000 \\ 0.0013 & 0.0000 & 0.0000 & 0.0012 & -0.0000 & 0.0000 \\ -0.1916 & 0.0001 & 0.0012 & 0.2290 & -0.0005 & 0.0010 \\ -0.0001 & -0.0000 & -0.0000 & -0.0005 & 0.0000 & 0.0000 \\ -0.0018 & -0.0000 & 0.0000 & 0.0010 & 0.0000 & 0.0000 \end{bmatrix} \\
G_{12}^e &= 10^4 \cdot \begin{bmatrix} -7.0520 & 0.0129 & 0.0188 & -2.1171 & -0.0108 & 0.0007 \\ -0.0128 & 0.0000 & 0.0000 & 0.0056 & 0.0000 & -0.0000 \\ -0.0246 & 0.0000 & 0.0000 & -0.0003 & -0.0000 & -0.0000 \\ -2.4389 & -0.0055 & 0.0008 & 2.0645 & 0.0043 & -0.0105 \\ 0.0107 & 0.0000 & -0.0000 & -0.0043 & -0.0000 & 0.0000 \\ -0.0003 & -0.0000 & -0.0000 & 0.0081 & 0.0000 & -0.0000 \end{bmatrix} \\
G_{21}^e &= 10^4 \cdot \begin{bmatrix} -7.0520 & -0.0128 & -0.0246 & -2.4389 & 0.0107 & -0.0003 \\ 0.0129 & 0.0000 & 0.0000 & -0.0055 & 0.0000 & -0.0000 \\ 0.0188 & 0.0000 & 0.0000 & 0.0008 & -0.0000 & -0.0000 \\ -2.1171 & 0.0056 & -0.0003 & 2.0645 & -0.0043 & 0.0081 \\ -0.0108 & 0.0000 & -0.0000 & 0.0043 & -0.0000 & 0.0000 \\ 0.0007 & -0.0000 & -0.0000 & -0.0105 & 0.0000 & -0.0000 \end{bmatrix} \\
G_{22}^e &= 10^5 \cdot \begin{bmatrix} 1.1372 & -0.0007 & -0.0009 & -0.1881 & 0.0002 & 0.0022 \\ -0.0007 & 0.0000 & 0.0000 & -0.0001 & -0.0000 & -0.0000 \\ -0.0009 & 0.0000 & 0.0000 & -0.0011 & -0.0000 & 0.0000 \\ -0.1881 & -0.0001 & -0.0011 & 0.1934 & 0.0005 & -0.0012 \\ 0.0002 & -0.0000 & -0.0000 & 0.0005 & 0.0000 & 0.0000 \\ 0.0022 & -0.0000 & 0.0000 & -0.0012 & 0.0000 & 0.0000 \end{bmatrix}
\end{aligned}$$

2.2 Task 2B - Pre-Buckling Analysis

If the applied loads increase monotonically, we can determine the critical buckling load parameter λ by identifying the smallest eigenvalue $\lambda_i = \lambda_1$ that satisfies Equation 2.4. Therefore, λ_1 acts as a scaling factor for the in-plane reference loads ($P^{(R)}$) used in the in-plane analysis. For the problem in hand, $\lambda_1 = 132.86$, which means that the applied in-plane compressive load could be increased by this amount for buckling to occur. The other output of the solution of the eigenvalue problem is the eigenvector, plotted below:

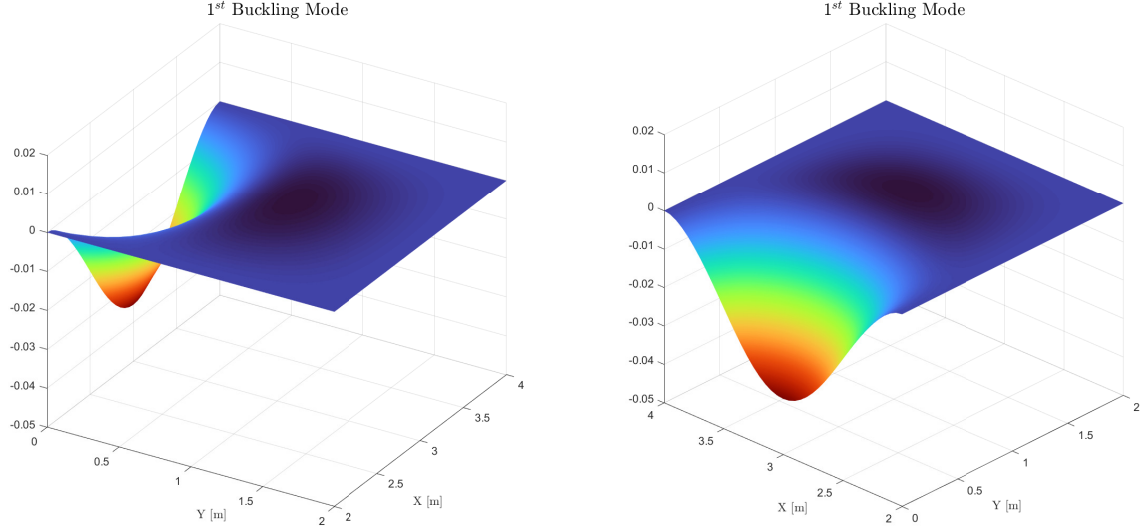


Figure 2.1: Smallest Positive Eigenmode

2.3 Results and Discussion

The eigenvalues of a Linearized Pre-Buckling Analysis represent a factor of safety or critical load multiplier for the applied loads that induce instability. For this reason, one common practice when running this type of analysis is to define a unit load, so that the eigenvalues obtained will represent the critical buckling load, often denoted as P_{cr} . If the analyst chooses to input the actual load values, the output from the LBA means "how many times the load can be increased before structural failure. Therefore, any value above 1.0 means that the structure is safe against instability - if the structure is assumed to behave linear elastically and geometrically, and no geometric imperfections are present. If one of these assumptions can't be fulfilled, then a nonlinear buckling analysis should be carried out. For the problem in hand, all these assumptions are valid and therefore the structure might be regarded as safe against instability by a load multiplier or factor of safety of ≈ 133 .

A

Appendix: Code listing

A.1 Task 1B, Kirchhoff Element Routine

```
1 function [Kww, Kaa, fwe_ext, fae_ext] = Full_KQuad_Func(ex,ey,ep,D,q,eq)
2
3 % -----
4 % PURPOSE
5 % Calculate the stiffness matrix for a Kirchhoff plane stress quad
  element
6 %
7 %
8 % INPUT:  ex = [x1 x2 x3 x4]      element coordinates
9 %         ey = [y1 y2 y3 y4]
10 %
11 %         ep = [ptype t]          ptype: analysis type      1 for Plane
  Stres
12 %                                         t: thickness
13 %
14 %         q = scalar              Uniform pressure - out of plane
15 %
16 %         D              constitutive matrix      Obtained by
  Hooke
17 %
18 %         eq = [bx;              bx: body force x-dir
19 %              by]              by: body force y-dir
20 %
21 % OUTPUT: Kww : element bending stiffness matrix (12 x 12)
22 %         Kaa : element stiffness matrix (8 x 8)
23 %         fwe_ext: element external force matrix, out-of-plane (12 x 1)
24 %         fae_ext: element external force matrix, in-plane (8 x 1)
25
26 ptype = ep(1);
27 h = ep(2);
28
29 Dbar = D*h^3/12;
30
31 % Gauss points
32 H_v = ones(1,4);
33 xi_v = [-1/sqrt(3) -1/sqrt(3) 1/sqrt(3) 1/sqrt(3);
34         -1/sqrt(3) 1/sqrt(3) -1/sqrt(3) 1/sqrt(3)];
35
36 Kww = zeros(12,12);
37 Kaa = zeros(8);
38 fwe_ext = zeros(12,1);
39 fae_ext = zeros(8,1);
40
```

```

41 for gp=1:4
42     Hgp = H_v(gp);
43     xin = xi_v(:,gp);
44     detFisop = detFisop_4node_func(xin, [ ex(1) ey(1) ]', ...
45                                         [ ex(2) ey(2) ]', ...
46                                         [ ex(3) ey(3) ]', ...
47                                         [ ex(4) ey(4) ]'');
48
49     [Be, Ne] = Be_Quad_func(xin, [ ex(1) ey(1) ]', ...
50                                  [ ex(2) ey(2) ]', ...
51                                  [ ex(3) ey(3) ]',...
52                                  [ ex(4) ey(4) ]'');
53
54     Bastn = Bast_kirchoff_func(xin, [ ex(1) ey(1) ]', ...
55                                     [ ex(2) ey(2) ]', ...
56                                     [ ex(3) ey(3) ]',...
57                                     [ ex(4) ey(4) ]'');
58
59     N_oop = Nk_func(xin,[ ex(1) ey(1) ]',[ ex(2) ey(2) ]',...
60                    [ ex(3) ey(3) ]',[ ex(4) ey(4) ]'');
61
62     fwe_ext = fwe_ext + N_oop'*q*detFisop*Hgp;
63     fae_ext = fae_ext + Ne'*eq*detFisop*h*Hgp;
64
65     Kaa = Kaa + Be'*D*Be*detFisop*h*Hgp;
66     Kww = Kww + Bastn'*Dbar*Bastn*detFisop*Hgp;
67 end

```

A.2 Task 1C, Solve for Displacements

```

1 clear
2 close all
3 clc
4
5 % ***** Unit scheme SI: N m kg s Pa
6 % ***** %
7 %% Property
8
9 E = 210e9; % Young's Modulus
10 nu = 0.3; % Poisson's Ration
11 Sy = 450e6; % Yield Strength
12 G_modulus = E/(2*(1+nu)); % Transverse Modulus
13 G = G_modulus*eye(2);
14 D = hooke(1,E,nu); % Material Matrix
15 rho = 1e3; % Water density for hydrostatic pressure
16 h = 10e-3; % Thickness
17 ep = [1 h];
18 %% Meshing
19
20 xmin = 2; xmax = 4; ymin = 0; ymax = 2;
21 nelx = 40; nely = 40;
22 [mesh, coord, Edof_ip, Edof_op] = rectMesh(xmin, xmax, ymin, ymax, nelx,
23 nely);
24
25 % Extract x and y coordinates for each node in mesh
26 Ex = reshape(coord(mesh, 1), 4, [])'; % Get x-coordinates from coord
27 % using mesh indices
28 Ey = reshape(coord(mesh, 2), 4, [])'; % Get y-coordinates from coord
29 % using mesh indices
30
31 % Edof arrays for each edge of the square
32 LS_nodes = find(abs(coord(:,1)-xmin)<eps);
33 RS_nodes = find(abs(coord(:,1)-xmax)<eps);
34 BS_nodes = find(abs(coord(:,2)-ymax)<eps);
35 TS_nodes = find(abs(coord(:,2)-ymin)<eps);
36 TT_nodes = find(abs(coord(TS_nodes,1) >= 2.5 & abs(coord(TS_nodes,1)) <=
37 3.5));
38
39 Edof_ip = full(Edof_ip);
40 Edof_op = full(Edof_op);
41
42 % Find all DoFs and count it for in-plane and out-of-plane
43 ndof_ip = numel(unique(Edof_ip));
44 ndof_op = numel(unique(Edof_op));
45 nnode = ndof_ip/2; % Number of nodes
46
47 Dof_ip = zeros(nnode,2); % Rows are nodes and columns are dofs
48 % (x,y)
49 Dof_op = zeros(nnode,3); % Rows are nodes and columns are dofs
50 % (w,th1,th2)
51
52 % Populate Dof matrices
53 for i=1:nnode
54     Dof_ip(i,1) = 2*i-1;

```

```

49     Dof_ip(i,2) = 2*i;
50     Dof_op(i,1) = 3*i-2;
51     Dof_op(i,2) = 3*i-1;
52     Dof_op(i,3) = 3*i;
53 end
54
55 % Retrieve the dofs of edges of interest for both in-plane and out-of-
    plane
56 RS_ip = Dof_ip(RS_nodes,:);
57 TS_ip = Dof_ip(TS_nodes,:);
58 LS_ip = Dof_ip(LS_nodes,:);
59 BS_ip = Dof_ip(BS_nodes,:);
60 TT_ip = Dof_ip(TT_nodes,:); % Top traction in-plane DOFs
61 RS_op = Dof_op(RS_nodes,:);
62 LS_op = Dof_op(LS_nodes,:);
63 BS_op = Dof_op(BS_nodes,:);
64
65 nelelem = size(Edof_ip,1);
66 %% Load and Boundary Conditions
67
68 l = 1; % Length of the edge
69 g = 9.806; % Gravity
70 P = (150*g)/(l*h); % Uniform Pressure
71 eq = g*[0; 0]; % Body Forces
72 faa = zeros(ndof_ip,1); % Global In-Plane Load vector
73 fww = zeros(ndof_op,1); % Global Out-of-Plane Load vector
74
75 % Calculate Pressure load for In-Plane
76 for i = 1:size(TT_nodes, 1)-1 % For the number of elements on
    the Top T edge (# of nodes -1)
77     pos1 = coord(TT_nodes(i,:),:);
78     pos2 = coord(TT_nodes(i+1,:),:);
79     Ep = pressure_load(pos1, pos2, P, h); % Element Pressure
80     dofs = reshape(TT_ip(i:i+1,:)', 1, []); % Flattens the dofs matrix
81     faa(dofs) = faa(dofs) + Ep;
82 end
83
84 % Create BC vector for In-Plane
85 ux_0 = [LS_ip(:,1); BS_ip(:,1); RS_ip(:,1)]; % Creates a vector with
    all DOFs that will have ux = 0
86 uy_0 = [LS_ip(:,2); BS_ip(:,2); RS_ip(:,2)]; % Creates a vector with
    all DOFs that will have uy = 0
87 pdis_ip = unique([ux_0; uy_0]); % Concatenates
    prescribed DOFs in a vector
88 pd_0_ip = zeros(size(pdis_ip,1),1); % Creates a vector of
    zeroes that correspond to the prescribed displacements
89 bc_ip = [pdis_ip pd_0_ip]; % Column 1 is DoF,
    Column 2 is prescribed value
90
91 % Create BC vector for Out-Plane
92 uz_0 = [LS_op(:,1); BS_op(:,1); RS_op(:,1)]; % w = 0
93 tx_0 = [LS_op(:,2); BS_op(:,2); RS_op(:,2)]; % theta_x = 0
94 ty_0 = [LS_op(:,3); BS_op(:,3); RS_op(:,3)]; % theta_y = 0
95 pdis_op = unique([uz_0; tx_0; ty_0]); % Concatenates
    prescribed DOFs in a vector
96 pd_0_op = zeros(size(pdis_op,1),1); % Creates a vector of
    zeroes that correspond to the prescribed displacements
97 bc_op = [pdis_op pd_0_op]; % Column 1 is DoF,

```

```

    Column 2 is prescribed value
98
99 %% Stiffness Assembly + Out-of-Plane Loads
100
101 % Assembly Global Kaa and Kww Matrices and fww matrix (out-of-plane
    pressure)
102 Kaa = zeros(ndof_ip, ndof_ip);      % Global In-Plane Stiffness Matrix
103 Kww = zeros(ndof_op, ndof_op);      % Global Bending Stiffness Matrix
104
105 for i=1:nelem
106     edof_ip = Edof_ip(i,2:end);
107     edof_op = Edof_op(i,2:end);
108     ex = Ex(i,:);
109     ey = Ey(i,:);
110     yel = sum(ey)/size(ey,2);
111     q = rho*g*yel;
112     [Kew, Kea, fwe_ext, ~] = Full_KQuad_Func(ex,ey,ep,D,q,eq);
113     Kaa(edof_ip, edof_ip) = Kaa(edof_ip, edof_ip) + Kea;
114     Kww(edof_op, edof_op) = Kww(edof_op, edof_op) + Kew;
115     fww(edof_op) = fww(edof_op) + fwe_ext;
116 end
117
118 %% Solve In-Plane Problem
119
120 % Solution of the [Kaa]{Ua} = {fa} system
121 fdof_ip = 1:ndof_ip;                % Vector that contains all
    degrees of freedom
122 U_ip = zeros(ndof_ip,1);            % Displacement vector
123 pdof_ip = bc_ip(:,1);                % Retrieves dofs that have a
    prescribed value
124 dp_ip = bc_ip(:,2);                 % Retrieves prescribed value
    of dofs
125 fdof_ip(pdof_ip) = [];              % DOFs with U = 0 are
    crossed-out
126
127 % Solve
128 s_ip = Kaa(fdof_ip,fdof_ip)\(faa(fdof_ip)-Kaa(fdof_ip,pdof_ip)*dp_ip); %
    Condensed problem to solve for unknown displacements s = [Kff]^-1 *
    (ff - Kfc*ac)
129
130 % Populate displacements vector and calculate reaction force
131 U_ip(pdof_ip) = dp_ip;
132 U_ip(fdof_ip) = s_ip;
133 Ra = Kaa*U_ip - faa;
134
135 %% Solve Out-of-Plane Problem
136
137 % Solution of the [Kww]{Uw} = {fw} system
138 fdof_op = 1:ndof_op;                % Vector that contains all
    degrees of freedom
139 U_op = zeros(ndof_op,1);            % Displacement vector
140 pdof_op = bc_op(:,1);                % Retrieves dofs that have a
    prescribed value
141 dp_op = bc_op(:,2);                 % Retrieves prescribed value
    of dofs
142 fdof_op(pdof_op) = [];              % DOFs with U = 0 are
    crossed-out
143

```

```

144 Kww_FF = Kww(fdof_op,fdof_op);
145 fww_FF = fww(fdof_op);
146 % Solve
147 s_op = Kww_FF\(fww_FF-Kww(fdof_op,pdof_op)*dp_op);
148
149 % Populate displacements vector and calculate reaction force
150 U_op(pdof_op) = dp_op;
151 U_op(fdof_op) = s_op;
152 Ro = Kww*U_op - fww;
153 %% Extracting Displacements and Stresses
154
155 Ux = U_ip(1:2:end);
156 Uy = U_ip(2:2:end);
157 Uw = U_op(1:3:end);
158 Tx = U_op(2:3:end);
159 Ty = U_op(3:3:end);
160 E_ip = extract_ed(Edof_ip, U_ip);
161 E_op = extract_ed(Edof_op, U_op);
162
163 %% Plotting Displacements
164
165 % Compute deformed nodal coordinates
166 x_def_ip = Ex + 600000*E_ip(:,1:2:end);
167 y_def_ip = Ey + 600000*E_ip(:,2:2:end);
168
169 E_ip_mag = sqrt(sum(E_ip.^2, 2));
170
171 % In plane
172 figure('Position', [900, 100, 720, 720]); % [left, bottom, width, height]
173 fill(x_def_ip', y_def_ip', E_ip_mag);
174 colormap(turbo);
175 c = colorbar;
176 xlabel('X [m]', 'Interpreter', 'latex');
177 ylabel('Y [m]', 'Interpreter', 'latex');
178 ylabel(c, 'Displacements [m]', Interpreter='latex', FontSize=14)
179 title('In-Plane Displacements',Interpreter='latex', FontSize=16);
180 set(gca, 'YDir', 'reverse')
181
182 % Compute deformed nodal coordinates
183 x_def = coord(:,1);% + Ux;
184 y_def = coord(:,2);% + Uy;
185 z_def = Uw; % out-of-plane displacement
186
187 % Reshape the deformed coordinates to a grid (nely+1 by nelx+1)
188 x_def_grid = reshape(x_def, [nely+1, nelx+1]);
189 y_def_grid = reshape(y_def, [nely+1, nelx+1]);
190 z_def_grid = reshape(z_def, [nely+1, nelx+1]);
191
192 % Plot the deformed configuration as a 3D surface
193 figure('Position', [100, 100, 720, 720]); % [left, bottom, width, height]
194 surf(x_def_grid, y_def_grid, z_def_grid);
195 colormap turbo;
196 colorbar_handle = colorbar;
197 xlabel('X [m]', Interpreter='latex');
198 ylabel('Y [m]', Interpreter='latex');
199 ylabel(colorbar_handle, 'Displacement [m]', 'Interpreter', 'latex',

```

```
    FontSize=14);          % Set the label for the colorbar
200 xlabel('Displacement [m]', Interpreter='latex');
201 title('Out of Plane Displacements',Interpreter='latex', FontSize=16);
202 zlim([min(z_def_grid(:)) 0.035]); % Adjust the upper limit of z axis
203 view(3);
```

A.3 Task 1D, Stress Function

```

1 function [e_ip_sig] = K_inplane_S(aue, awe, ex, ey, z, D)
2 % Calculates In-Plane Stress for Kirchhoff plate element in Voigt Form
3
4 % Gauss points
5 ngp = 4;
6 xi_v = [-1/sqrt(3) -1/sqrt(3) 1/sqrt(3) 1/sqrt(3);
7         -1/sqrt(3) 1/sqrt(3) -1/sqrt(3) 1/sqrt(3)];
8
9 Sig_ip_e = zeros(3,ngp); %          GP1 GP2 GP3 GP4          -
10      Sigma_inplane_element
11                                % S11
12                                % S22
13                                % S12
14 for j=1:ngp
15     xin = xi_v(:,j);          % Gauss-Point Coordinate
16
17     [Be, ~] = Be_Quad_func(xin, [ ex(1) ey(1) ]', ...
18                                [ ex(2) ey(2) ]', ...
19                                [ ex(3) ey(3) ]',...
20                                [ ex(4) ey(4) ]');
21
22     Bastn = Bast_kirchoff_func(xin, [ ex(1) ey(1) ]', ...
23                                   [ ex(2) ey(2) ]', ...
24                                   [ ex(3) ey(3) ]',...
25                                   [ ex(4) ey(4) ]');
26
27     Sig_ip_e(:,j) = D * (Be*aue' - z*Bastn*awe');
28 end
29
30 e_ip_sig = mean(Sig_ip_e,2); % Averages Gauss Point stresses -
    element_inplane_sigma (averaged out) 3x1 vector

```


A.4 Task 2A, Geometric Stiffness Matrix Function

```

1 function [Gr] = Kirch_G(ex, ey, t, eSS)
2 % -----
3 % PURPOSE
4 %   Calculate the Geometric matrix for a Kirchhoff Plate element
5 %
6 %
7 %   INPUT:   ex = [x1 x2 x3 x4]           element coordinates
8 %            ey = [y1 y2 y3 y4]
9 %
10 %            t = scalar                    t: thickness
11 %
12 %            D                                constitutive matrix      Obtained by
13 %            Hooke
14 %            eSS = [Sxx;                    Element Stress State (output from
15 %                  K_inplane_S function)
16 %                  Syy;
17 %                  Sxy];
18 %   OUTPUT: Gr : element geometric matrix (12 x 12)
19
20 Nsec = t*[eSS(1) eSS(3);
21           eSS(3) eSS(2)];
22
23 Gr = zeros(12,12);
24
25
26 % Gauss points
27 ngp = 9;
28 H_v = [25/81 25/81 25/81 25/81 40/81 40/81 40/81 40/81 64/81];
29 xi_v = [-sqrt(3/5) sqrt(3/5) -sqrt(3/5) sqrt(3/5) 0 0
30         -sqrt(3/5) sqrt(3/5) 0;
31         -sqrt(3/5) -sqrt(3/5) sqrt(3/5) sqrt(3/5) -sqrt(3/5) sqrt(3/5)
32         0 0 0];
33
34 for gp=1:ngp
35     Hgp = H_v(gp);
36     xin = xi_v(:,gp);
37
38     detFisop = detFisop_4node_func(xin, [ ex(1) ey(1) ]', ...
39                                         [ ex(2) ey(2) ]', ...
40                                         [ ex(3) ey(3) ]', ...
41                                         [ ex(4) ey(4) ]'));
42
43     Be_kirch = Be_kirch_func(xin,[ ex(1) ey(1) ]', ...
44                                [ ex(2) ey(2) ]', ...
45                                [ ex(3) ey(3) ]',...
46                                [ ex(4) ey(4) ]'));
47
48     Gr = Gr + Be_kirch*Nsec*Be_kirch'*Hgp*detFisop;
49 end

```

A.5 Task 2B, Buckling Problem

```

1 clear
2 close all
3 clc
4
5 % ***** Unit scheme SI: N m kg s Pa
6 % ***** %
7 %% Property
8
9 E = 210e9; % Young's Modulus
10 nu = 0.3; % Poisson's Ration
11 Sy = 450e6; % Yield Strength
12 G_modulus = E/(2*(1+nu)); % Transverse Modulus
13 G = G_modulus*eye(2);
14 D = hooke(1,E,nu); % Material Matrix
15 rho = 1e3; % Water density for hydrostatic pressure
16 h = 10e-3; % Thickness
17 ep = [1 h];
18 %% Meshing
19
20 xmin = 2; xmax = 4; ymin = 0; ymax = 2;
21 nelx = 40; nely = 40;
22 [mesh, coord, Edof_ip, Edof_op] = rectMesh(xmin, xmax, ymin, ymax, nelx,
23 nely);
24
25 % Extract x and y coordinates for each node in mesh
26 Ex = reshape(coord(mesh, 1), 4, [])'; % Get x-coordinates from coord
27 % using mesh indices
28 Ey = reshape(coord(mesh, 2), 4, [])'; % Get y-coordinates from coord
29 % using mesh indices
30
31 % Edof arrays for each edge of the square
32 LS_nodes = find(abs(coord(:,1)-xmin)<eps);
33 RS_nodes = find(abs(coord(:,1)-xmax)<eps);
34 BS_nodes = find(abs(coord(:,2)-ymax)<eps);
35 TS_nodes = find(abs(coord(:,2)-ymin)<eps);
36 TT_nodes = find(abs(coord(TS_nodes,1) >= 2.5 & abs(coord(TS_nodes,1)) <=
37 3.5));
38
39 Edof_ip = full(Edof_ip);
40 Edof_op = full(Edof_op);
41
42 % Find all DoFs and count it for in-plane and out-of-plane
43 ndof_ip = numel(unique(Edof_ip));
44 ndof_op = numel(unique(Edof_op));
45 nnode = ndof_ip/2; % Number of nodes
46
47 Dof_ip = zeros(nnode,2); % Rows are nodes and columns are dofs
48 % (x,y)
49 Dof_op = zeros(nnode,3); % Rows are nodes and columns are dofs
50 % (w,th1,th2)
51
52 % Populate Dof matrices
53 for i=1:nnode
54     Dof_ip(i,1) = 2*i-1;

```

```

49     Dof_ip(i,2) = 2*i;
50     Dof_op(i,1) = 3*i-2;
51     Dof_op(i,2) = 3*i-1;
52     Dof_op(i,3) = 3*i;
53 end
54
55 % Retrieve the dofs of edges of interest for both in-plane and out-of-
    plane
56 RS_ip = Dof_ip(RS_nodes,:);
57 TS_ip = Dof_ip(TS_nodes,:);
58 LS_ip = Dof_ip(LS_nodes,:);
59 BS_ip = Dof_ip(BS_nodes,:);
60 TT_ip = Dof_ip(TT_nodes,:); % Top traction in-plane DOFs
61 RS_op = Dof_op(RS_nodes,:);
62 LS_op = Dof_op(LS_nodes,:);
63 BS_op = Dof_op(BS_nodes,:);
64
65 nelelem = size(Edof_ip,1);
66 %% Load and Boundary Conditions
67
68 l = 1; % Length of the edge
69 g = 9.806; % Gravity
70 P = (150*g)/(l*h); % Uniform Pressure
71
72 eq = g*[0; 0]; % Body Forces
73 faa = zeros(ndof_ip,1); % Global In-Plane Load vector
74 fww = zeros(ndof_op,1); % Global Out-of-Plane Load vector
75
76 % Calculate Pressure load for In-Plane
77 for i = 1:size(TT_nodes, 1)-1 % For the number of elements on
    the Top T edge (# of nodes -1)
78     pos1 = coord(TT_nodes(i,:),:);
79     pos2 = coord(TT_nodes(i+1,:),:);
80     Ep = pressure_load(pos1, pos2, P, h); % Element Pressure
81     dofs = reshape(TT_ip(i:i+1,:), 1, []); % Flattens the dofs matrix
82     faa(dofs) = faa(dofs) + Ep;
83 end
84
85 % Create BC vector for In-Plane
86 ux_0 = [LS_ip(:,1); BS_ip(:,1); RS_ip(:,1)]; % Creates a vector with
    all DOFs that will have ux = 0
87 uy_0 = [LS_ip(:,2); BS_ip(:,2); RS_ip(:,2)]; % Creates a vector with
    all DOFs that will have uy = 0
88 pdis_ip = unique([ux_0; uy_0]); % Concatenates
    prescribed DOFs in a vector
89 pd_0_ip = zeros(size(pdis_ip,1),1); % Creates a vector of
    zeroes that correspond to the prescribed displacements
90 bc_ip = [pdis_ip pd_0_ip]; % Column 1 is DoF,
    Column 2 is prescribed value
91
92 % Create BC vector for Out-Plane
93 uz_0 = [LS_op(:,1); BS_op(:,1); RS_op(:,1)]; % w = 0
94 tx_0 = [LS_op(:,2); BS_op(:,2); RS_op(:,2)]; % theta_x = 0
95 ty_0 = [LS_op(:,3); BS_op(:,3); RS_op(:,3)]; % theta_y = 0
96 pdis_op = unique([uz_0; tx_0; ty_0]); % Concatenates
    prescribed DOFs in a vector
97 pd_0_op = zeros(size(pdis_op,1),1); % Creates a vector of
    zeroes that correspond to the prescribed displacements

```

```

98 bc_op = [pdis_op pd_0_op]; % Column 1 is DoF,
    Column 2 is prescribed value
99
100 %% Stiffness Assembly + Out-of-Plane Loads
101
102 % Assembly Global Kaa and Kww Matrices and fww matrix (out-of-plane
    pressure)
103 Kaa = zeros(ndof_ip, ndof_ip); % Global In-Plane Stiffness Matrix
104 Kww = zeros(ndof_op, ndof_op); % Global Bending Stiffness Matrix
105
106 for i=1:nelem
107     edof_ip = Edof_ip(i,2:end);
108     edof_op = Edof_op(i,2:end);
109     ex = Ex(i,:);
110     ey = Ey(i,:);
111     yel = sum(ey)/size(ey,2);
112     q = rho*g*yel;
113     [Kew, Kea, fwe_ext, ~] = Full_KQuad_Func(ex,ey,ep,D,q,eq);
114     Kaa(edof_ip, edof_ip) = Kaa(edof_ip, edof_ip) + Kea;
115     Kww(edof_op, edof_op) = Kww(edof_op, edof_op) + Kew;
116     fww(edof_op) = fww(edof_op) + fwe_ext;
117 end
118
119 %% Solve In-Plane Problem
120
121 % Solution of the [Kaa]{Ua} = {fa} system
122 fdof_ip = 1:ndof_ip; % Vector that contains all
    degrees of freedom
123
124 U_ip = zeros(ndof_ip,1); % Displacement vector
125 pdof_ip = bc_ip(:,1); % Retrieves dofs that have a
    prescribed value
126 dp_ip = bc_ip(:,2); % Retrieves prescribed value
    of dofs
127 fdof_ip(pdof_ip) = []; % DOFs with U = 0 are
    crossed-out
128
129 fdof_op = 1:ndof_op; % Vector that contains all
    degrees of freedom
130 pdof_op = bc_op(:,1); % Retrieves dofs that have a
    prescribed value
131 fdof_op(pdof_op) = []; % DOFs with U = 0 are
    crossed-out
132 Kww_FF = Kww(fdof_op,fdof_op);
133
134 % Solve
135 s_ip = Kaa(fdof_ip,fdof_ip)\(faa(fdof_ip)-Kaa(fdof_ip,pdof_ip)*dp_ip); %
    Condensed problem to solve for unknown displacements s = [Kff]^-1 *
    (ff - Kfc*ac)
136
137 % Populate displacements vector and calculate reaction force
138 U_ip(pdof_ip) = dp_ip;
139 U_ip(fdof_ip) = s_ip;
140 Ra = Kaa*U_ip - faa;
141
142 %% Extracting Displacements and Stresses
143
144 Ux = U_ip(1:2:end);

```

```

145 Uy = U_ip(2:2:end);
146
147 E_ip = extract_ed(Edof_ip, U_ip);
148
149 th_coord= [-h/2 0 h/2]; % Through thickness positions to
    evaluate stress
150
151 Svm_ip = zeros(nelem,3);
152 SS_ip = zeros(3,1,nelem);
153 % Loop over elements to obtain stresses
154 for iel = 1:nelem
155     aue = E_ip(iel,:);
156     awe = [0 0 0 0 0 0 0 0 0 0 0 0];
157     ex = Ex(iel,:);
158     ey = Ey(iel,:);
159     for k = 1:size(th_coord,2)
160         z = th_coord(k);
161         SS_ip_e = K_inplane_S(aue,awe,ex,ey,z,D); % Stress
    State In Plane [sxx syy sxy]'
162         if k==2
163             SS_ip(:,1,iel) = SS_ip_e; % Stores
    midplane stresses
164         end
165     end
166 end
167
168 %% Linearised Pre Buckling
169
170 % Compute Geometric Stiffness
171 % Initialize Global Geometric Matrix
172 Gr = zeros(ndof_op,ndof_op);
173 for iel=1:nelem
174     edof_op = Edof_op(iel,2:end);
175     ex = Ex(iel,:);
176     ey = Ey(iel,:);
177     eSS = SS_ip(:,1,iel);
178     Gr_e = Kirch_G(ex,ey,h,eSS);
179     Gr(edof_op, edof_op) = Gr(edof_op, edof_op) + Gr_e;
180 end
181
182 Gr_FF = Gr(fdof_op,fdof_op);
183
184 [V,D] = eigs(Kww_FF,-Gr_FF,1,'smallestabs');
185 V_w = zeros(ndof_op,1);
186 V_w(fdof_op,1) = V; % Retrieves the out of plane displacements
187 V_w = V_w(1:3:end);
188 z_buckl_grid = reshape(V_w, [nely+1, nelx+1]);
189
190
191 %% Plotting
192 % Compute deformed nodal coordinates
193 x_def = coord(:,1);% + Ux;
194 y_def = coord(:,2);% + Uy;
195
196 % Reshape the deformed coordinates to a grid (nely+1 by nelx+1)
197 x_def_grid = reshape(x_def, [nely+1, nelx+1]);
198 y_def_grid = reshape(y_def, [nely+1, nelx+1]);
199

```

```
200 % Plot the deformed buckling mode as a 3D surface
201 figure('Position', [100, 100, 720, 720]); % [left, bottom, width, height
    ]
202 surf(x_def_grid, y_def_grid, z_buckl_grid);
203 colormap turbo;
204 colormap(flipud(colormap));
205 colorbar_handle = colorbar;
206 xlabel('X [m]', Interpreter='latex');
207 ylabel('Y [m]', Interpreter='latex');
208 title('$1^{st}$ Buckling Mode',Interpreter='latex', FontSize=16);
209 zlim([-0.05 0.02]); % Adjust the upper limit of z axis
210 view(3);
211 set(gca, 'YDir', 'reverse')
212 rotate3d on;
213 annotation('textbox', [0.15, 0.8, 0.1, 0.1], 'String', 'FoS = 133', ...
214     'FitBoxToText', 'on', 'BackgroundColor', 'white');
```

B

Appendix: Support Code Listing

B.1 Kirchhoff Bending B-Matrix

```
1 %% Compute Kirchhoff Plate Element N_oop function
2 xi = sym('xi',[2,1],'real');
3
4 % Define base functions for isoparam quadrilateral element
5 N1 = 0.25*(xi(1) - 1)*(xi(2) - 1); N2 = -0.25*(xi(1) + 1)*(xi(2) - 1);
6 N3 = 0.25*(xi(1) + 1)*(xi(2) + 1); N4 = -0.25*(xi(1) - 1)*(xi(2) + 1);
7
8 % Introduce node positions
9 xe1 = sym('xe1',[2,1],'real');
10 xe2 = sym('xe2',[2,1],'real');
11 xe3 = sym('xe3',[2,1],'real');
12 xe4 = sym('xe4',[2,1],'real');
13
14 % Introduce spatial coordinate as fcn of isoparam. coord.
15 x=N1*xe1+N2*xe2+N3*xe3+N4*xe4;
16
17 % Compute Jacobian
18 Fisop=jacobian(x,xi);
19 invFisop=simplify(inv(Fisop));
20 detFisop = simplify(det(Fisop));
21
22 % Create Matlab functions for later use
23 matlabFunction(invFisop,'File','invFisop_4node_func','Vars',{xi,xe1,xe2,
    xe3,xe4});
24 matlabFunction(detFisop,'File','detFisop_4node_func','Vars',{xi,xe1,xe2,
    xe3,xe4});
25
26 % Define P matrix, alpha coefficients and w(xi)
27 P(xi)=[1 xi(1) xi(2) xi(1)^2 xi(1)*xi(2) xi(2)^2 ... %6
28 xi(1)^3 xi(1)^2*xi(2) xi(1)*xi(2)^2 xi(2)^3 ...
29 xi(1)^3*xi(2) xi(1)*xi(2)^3];
30 alpha = sym('alpha',[12,1],'real');
31 w(xi)=P(xi(1),xi(2))*alpha;
32
33 % Differentiate P with respect to x, y. Use chain rule.
34 dPdx1 = sym('dPxi',[12,2],'real')
35 dPdx11(xi)=diff(P(xi(1),xi(2)),xi(1))
36 dPdx12(xi)=diff(P(xi(1),xi(2)),xi(2))
37 dPdx1(xi)=jacobian(P(xi(1),xi(2)),xi)*invFisop(:,1)
38 dPdx2(xi)=jacobian(P(xi(1),xi(2)),xi)*invFisop(:,2)
39
40 % Initialize A matrix
41 A = sym('A',[12,12],'real');
```

```

42
43 % Give points (corner nodes in isoparam element)
44 np=[-1 -1; -1 -1; -1 -1;...
45 1 -1; 1 -1; 1 -1;...
46 1 1; 1 1; 1 1; ...
47 -1 1; -1 1; -1 1];
48
49 % Define A matrix based on P and derivatives of P
50 for i=[1 4 7 10]
51     A(i,:)=P(np(i,1),np(i,2));
52 end
53
54 for i=[2 5 8 11]
55     A(i,:)=dPdx1(np(i,1),np(i,2));
56 end
57
58 for i=[3 6 9 12]
59     A(i,:)=dPdx2(np(i,1),np(i,2));
60 end
61
62 % Finally, solve for N and produce matlab function
63 N_oop=simplify(P*inv(A));
64 matlabFunction(N_oop,'File','Nk_func','Vars',{xi,xe1,xe2,xe3,xe4});
65
66 %% Compute Be_kirchoff and B* matrix
67
68 % Initialize dNdx
69 dNdx=sym('dNdx',[12,2],'real');
70
71 % Define dNdx by using the chain rule
72 dNdx(:,1) = diff(N_oop(xi(1),xi(2)),xi(1))*invFisop(1,1) + ...
73 diff(N_oop(xi(1),xi(2)),xi(2))*invFisop(2,1);
74 dNdx(:,2) = diff(N_oop(xi(1),xi(2)),xi(1))*invFisop(1,2) + ...
75 diff(N_oop(xi(1),xi(2)),xi(2))*invFisop(2,2);
76
77
78 % Now Bast can be computed
79 Bast(1,:) = diff(dNdx(:,1),xi(1))*invFisop(1,1) + ...
80 diff(dNdx(:,1),xi(2))*invFisop(2,1);
81 Bast(2,:) = diff(dNdx(:,2),xi(1))*invFisop(1,2) + ...
82 diff(dNdx(:,2),xi(2))*invFisop(2,2);
83 Bast(3,:) = 2*(diff(dNdx(:,1),xi(1))*invFisop(1,2) + ...
84 diff(dNdx(:,1),xi(2))*invFisop(2,2));
85
86 matlabFunction(dNdx,'File','Be_kirch_func','Vars',{xi,xe1,xe2,xe3,xe4});
87 matlabFunction(Bast,'File','Bast_kirchoff_func','Vars',{xi,xe1,xe2,xe3,
    xe4});

```


B.2 Quad Element B-Matrix

```

1 %% Compute Plane Stress Element N function
2 xi = sym('xi',[2,1],'real');
3
4 % Define base functions for isoparam quadrilateral element
5 N1 = 0.25*(xi(1) - 1)*(xi(2) - 1); N2 = -0.25*(xi(1) + 1)*(xi(2) - 1);
6 N3 = 0.25*(xi(1) + 1)*(xi(2) + 1); N4 = -0.25*(xi(1) - 1)*(xi(2) + 1);
7
8 % Define N-matrix for the element
9 Ne = [N1 0 N2 0 N3 0 N4 0;
10       0 N1 0 N2 0 N3 0 N4];
11
12 % Differentiate shape functions wrt isoparam. coordinates
13 dN1_dxi=gradient(N1,xi);
14 dN2_dxi=gradient(N2,xi);
15 dN3_dxi=gradient(N3,xi);
16 dN4_dxi=gradient(N4,xi);
17
18 % Introduce initial node positions
19 xe1 = sym('xe1',[2,1],'real');
20 xe2 = sym('xe2',[2,1],'real');
21 xe3 = sym('xe3',[2,1],'real');
22 xe4 = sym('xe4',[2,1],'real');
23
24 % Introduce spatial coordinate as fcn of isoparam. coord.
25 x=N1*xe1+N2*xe2+N3*xe3+N4*xe4;
26
27 % Compute Jacobian
28 Fisop=jacobian(x,xi);
29 detFisop = det(Fisop);
30
31 % Use chain rule to compute material derivatives
32 dN1_dx=simplify( inv(Fisop)*dN1_dxi );
33 dN2_dx=simplify( inv(Fisop)*dN2_dxi );
34 dN3_dx=simplify( inv(Fisop)*dN3_dxi );
35 dN4_dx=simplify( inv(Fisop)*dN4_dxi );
36
37 % Define B0-matrix of element
38 Be0= [dN1_dx(1), 0, dN2_dx(1), 0,
        dN3_dx(1), 0, dN4_dx(1), 0;
        0, dN1_dx(2), 0, dN2_dx(2),
        0, dN3_dx(2), 0, dN4_dx(2);
        dN1_dx(2), dN1_dx(1), dN2_dx(2), dN2_dx(1),
        dN3_dx(2), dN3_dx(1), dN4_dx(2), dN4_dx(1)];
39
40
41
42 matlabFunction(Be0, Ne, 'File','Be_Quad_func','Vars',{xi,xe1,xe2,xe3,xe4
    });

```